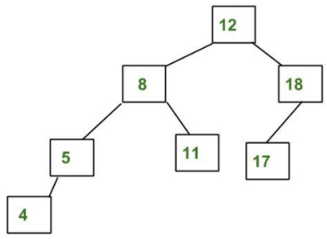
Introduction to AVL Tree

AVL tree is a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees cannot be more than one for all nodes.

An Example Tree that is an AVL Tree

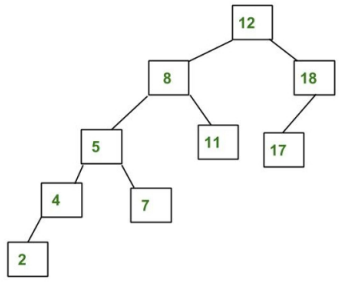
:



The above tree is AVL because differences between heights of left and right subtrees for every node is less than or equal to 1.

An Example Tree that is NOT an AVL Tree

:



The above tree is not AVL because differences between heights of left and right subtrees for 8 and 18 is greater than 1.

Why AVL Trees?

Most of the BST operations (e.g., search, max, min, insert, delete.. etc) take O(h) time where h is the height of the BST. The cost of these operations may become O(n) for a skewed Binary tree. If we make sure that the height of the tree remains O(Logn) after every insertion and deletion, then we can guarantee an upper bound of O(Logn) for all these operations. The height of an AVL tree is always O(Logn) where n is the number of nodes in the tree.

Insertion

To make sure that the given tree remains AVL after every insertion, we must augment the standard BST insert operation to perform some re-balancing. Following are two basic operations that can be performed to re-balance a BST without violating the BST property (keys(left) < key(root) < keys(right)).

1. Left Rotation
2. Right Rotation

```
T1, T2 and T3 are subtrees of the tree
rooted with y (on the left side) or x (on
the right side)
```

```
     y                               x
    / \     Right Rotation          /  \
   x   T3   - - - - - - - >        T1   y
  / \       < - - - - - - -            / \
 T1  T2     Left Rotation            T2  T3
```

```
Keys in both of the above trees follow the
following order:
 keys(T1) < key(x) < keys(T2) < key(y) < keys(T3)

So BST property is not violated anywhere.
```

Steps to follow for insertion

: Let the newly inserted node be **w**

1. Perform standard BST insert for w.
2. Starting from w, travel up and find the first unbalanced node. Let z be the first unbalanced node, y be the child of z that comes on the path from w to z and x be the grandchild of z that comes on the path from w to z.
3. Re-balance the tree by performing appropriate rotations on the subtree rooted with z. There can be 4 possible cases that need to be handled as x, y and z can be arranged in 4 ways. Following are the possible 4 arrangements:
   - y is left child of z and x is left child of y (Left Left Case)
   - y is left child of z and x is right child of y (Left Right Case)
   - y is right child of z and x is right child of y (Right Right Case)
   - y is right child of z and x is left child of y (Right Left Case)

Following are the operations to be performed in above mentioned 4 cases. In all of the cases, we only need to re-balance the subtree rooted with z and the complete tree becomes balanced as the height of subtree (After appropriate rotations) rooted with z becomes same as it was before insertion. (See

this
video lecture for proof)

a) Left Left Case

```
T1, T2, T3 and T4 are subtrees.
```

```
       z                                      y
      / \                                   /   \
     y   T4      Right Rotate (z)          x      z
    / \          - - - - - - - - ->      /  \    /  \
   x   T3                               T1  T2  T3  T4
  / \
T1   T2
```

b) Left Right Case

```
     z                               z                           x
    / \                            /   \                        /  \
   y   T4  Left Rotate (y)        x    T4  Right Rotate(z)    y      z
  / \      - - - - - - - - ->    /  \      - - - - - - - ->  / \    / \
T1   x                          y    T3                    T1  T2 T3  T4
    / \                        / \
  T2   T3                    T1   T2
```

c) Right Right Case

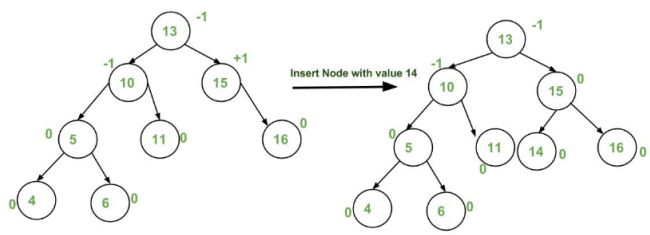
```
  z                                y
 /  \                            /   \
T1   y     Left Rotate(z)       z      x
```

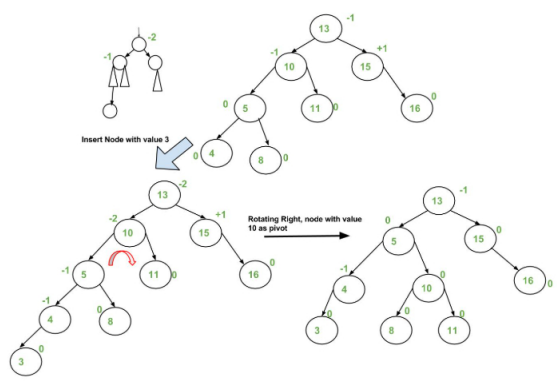
```
    /  \   - - - - - - - - ->    / \    / \
   T2   x                       T1  T2 T3  T4
       / \
     T3  T4
```

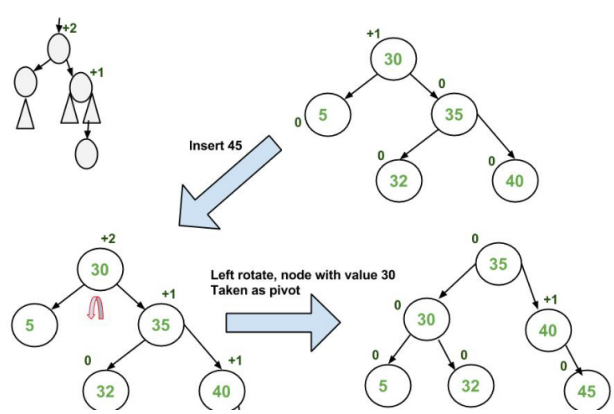
d) Right Left Case

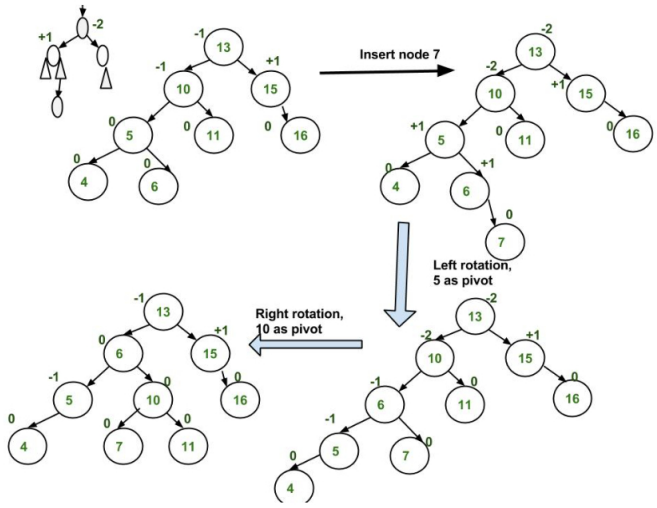
```
   z                            z                            x
  / \                          / \                          /  \
T1   y   Right Rotate (y)    T1   x      Left Rotate(z)   z      y
    / \  - - - - - - - - ->     /  \   - - - - - - - ->  / \    / \
   x   T4                      T2   y                  T1  T2  T3  T4
  / \                              /  \
T2   T3                           T3   T4
```

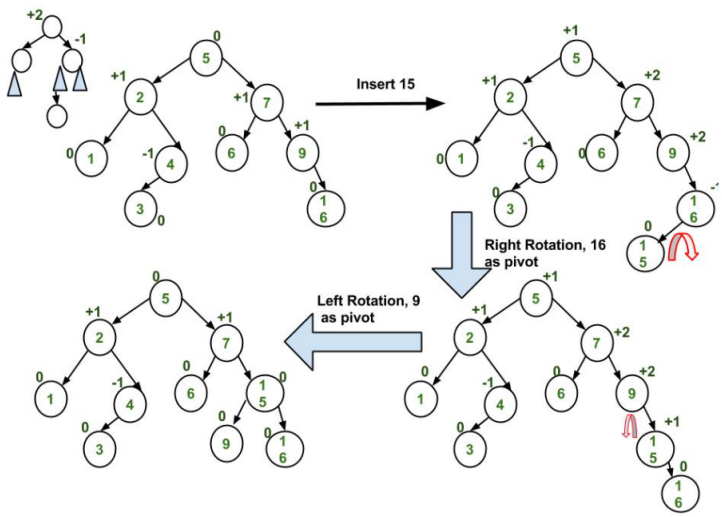
Insertion Examples:











Time Complexity:

The rotation operations (left and right rotate) take constant time as only a few pointers are being changed there. Updating the height and getting the balance factor also takes constant time. So the time complexity of AVL insert remains same as BST insert which is O(h) where h is the height of the tree. Since the AVL tree is balanced, the height is O(Logn). So time complexity of AVL insert is O(Logn).

Marked as Read

Report An Issue

If you are facing any issue on this page. Please let us know.

Articles
Videos
Problems
Quiz
Contest

Ceil in BST
Last Updated: 2022-12-29
Self Balancing BST
Last Updated: 2024-05-01
Introduction to AVL Tree
Last Updated: 2024-05-01
Red Black Tree
Last Updated: 2024-05-01
Applications of BST
Last Updated: 2022-12-18
Find Kth Smallest in BST

Articles Read
16 of 16 Complete. (100%)
Progress may take upto 2 hours to reflect.

« Prev
Next »